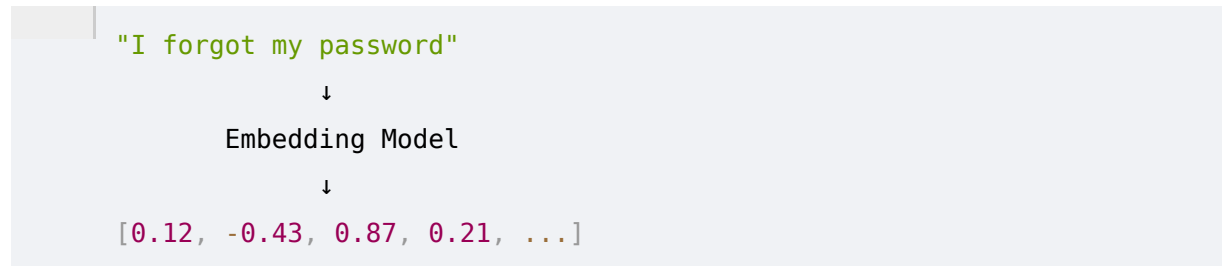


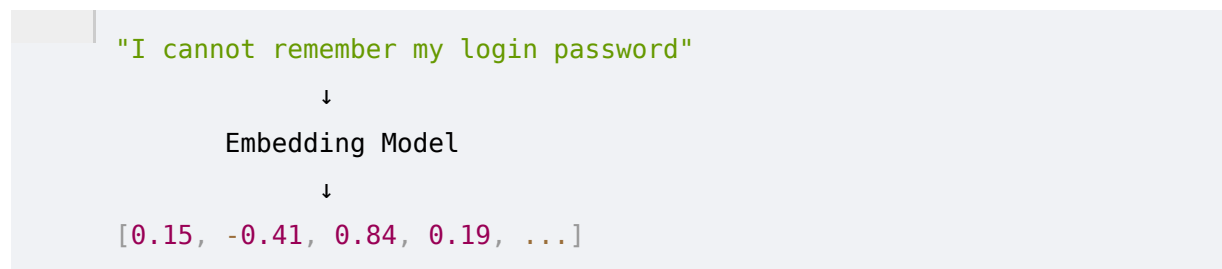
What is an embedding?

An **embedding** converts text into a numerical vector that represents the **meaning** of the text.

For example:



Another sentence:



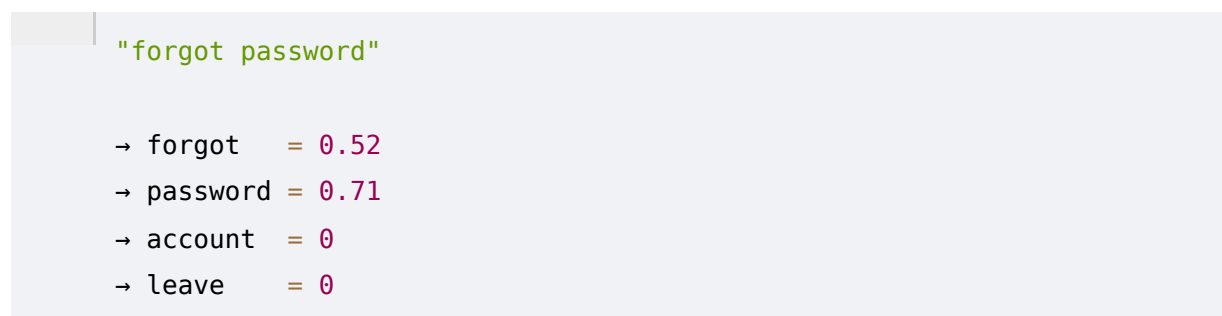
These vectors should be close together because the sentences have similar meanings.

Step 2 — TF-IDF vs Embedding

You already implemented TF-IDF, so this distinction is important.

TF-IDF

Your previous approach essentially represents a document according to **which words occur and how important those words are**.



This is a **sparse vector**.

For a vocabulary of 10,000 words, you might have:

```
[0, 0, 0, 0.52, 0, 0, 0.71, 0, 0, ...]
```

Embedding

An embedding model represents the **meaning/context** of the entire sentence.

```
"I forgot my password"
```

```
→ [0.12, -0.43, 0.87, 0.21, -0.16, ...]
```

This is a **dense vector**.

For example, an embedding model might produce 384, 768, or another fixed number of dimensions depending on the model.

Step 3 — Why embeddings are better for semantic search

Consider:

```
Document:
```

```
"If you forgot your password, use the password recovery option."
```

User:

```
"I cannot remember my login credentials."
```

There may be very little exact word overlap.

TF-IDF:

```
"I cannot remember my login credentials"
```

↓

```
"If you forgot your password..."
```

May produce a relatively weak similarity.

An embedding model can understand the relationship:

```
forgot  
remember  
password  
credentials  
login  
recovery
```

That's the main reason embeddings are useful for modern semantic search.

Your Stage 2 system will look like:



We need an **embedding model**.

```
Option A
Local embedding model
    ↓
No API required
    ↓
Good for learning
```

```
Option B
Cloud embedding API
    ↓
Very easy to use
    ↓
Requires API/service

Option C
ONNX embedding model
    ↓
```

For your learning project, I recommend a **local ONNX embedding model** first.

That lets you understand the entire pipeline:

```
C#
    ↓
ONNX Model
    ↓
Text
    ↓
Embedding
    ↓
Vector
    ↓
Cosine Similarity
```

without making your semantic-search project dependent on an external API.

Step 6 — One important concept before coding

Don't think of an embedding as:

```
word → number
```

That's closer to how traditional word-based approaches work.

Think of an embedding as:

```
sentence/document
    ↓
embedding model
    ↓
```

meaning representation
↓
vector

For example:

```
"I forgot my password"
```

and:

```
"I can't remember my login credentials"
```

are two different strings, but their embeddings should be relatively close in vector space.

That's the key idea behind modern semantic search.

Your next step

Before writing the C# implementation, we should understand **how the embedding model actually converts a sentence into a vector**, including:

- tokenizer
- tokens
- token IDs
- model input
- model output
- pooling
- final embedding vector

Once that is clear, we'll implement it in C# and generate embeddings for your existing **35 documents**.

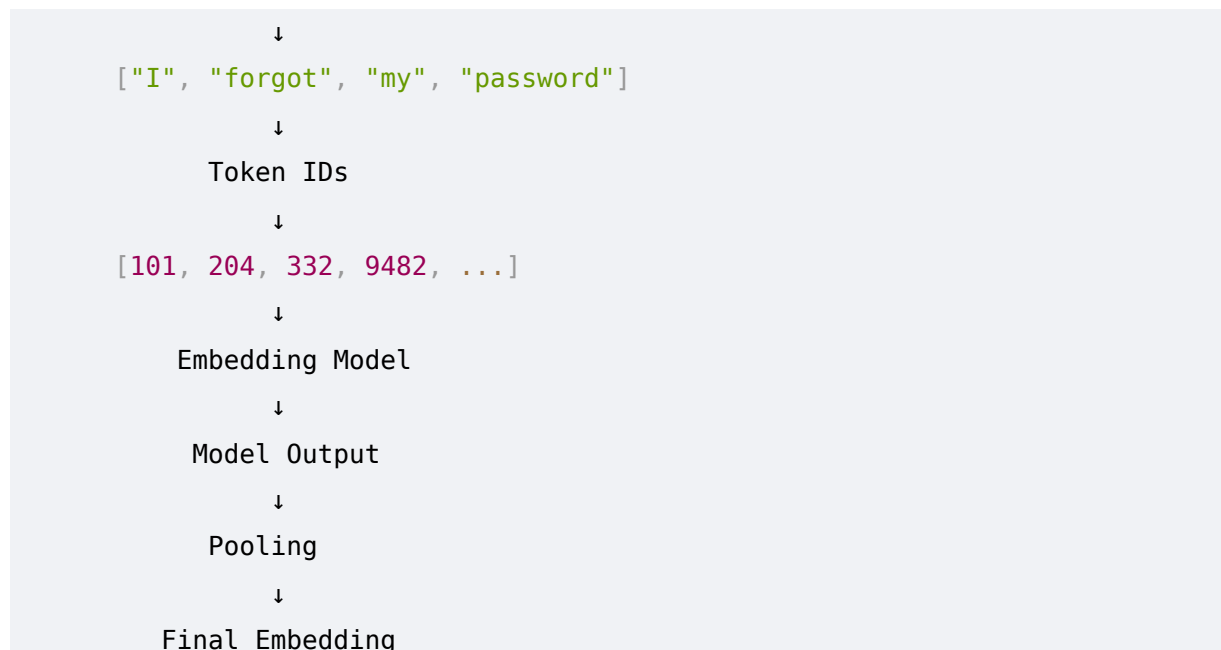
ok give me next step

Perfect. Now let's go one level deeper.

Step 2 — How text becomes an embedding vector

You need to understand this pipeline before implementing it in C#:

```
"I forgot my password"  
↓  
Tokenizer
```



1. Tokenization

An embedding model doesn't directly understand a C# string.

You give it:

```
string text = "I forgot my password";
```

The tokenizer converts the text into tokens.

Depending on the model, it could be represented roughly as:

```
I
forgot
my
password
```

But **don't assume one word = one token**. Modern tokenizers often split words into subwords.

For example:

```
"password"
```

could potentially be represented using one or more tokens depending on the tokenizer.

2. Token IDs

The tokenizer converts those tokens into numbers.

Conceptually:

```
"I"          → 101
"forgot"     → 204
"my"         → 332
"password"   → 9482
```

So the model actually receives numbers rather than your original string.

There are usually additional inputs such as:

```
input_ids
attention_mask
```

The exact inputs depend on the model.

3. The embedding model

Now those token IDs are passed into the neural network.

Conceptually:

```
Token IDs
  ↓
Neural Network
  ↓
Hidden Representations
```

The model generates a representation for each token.

For example, conceptually:

```
"I"          → [ . . . . ]
"forgot"     → [ . . . . ]
"my"         → [ . . . . ]
"password"   → [ . . . . ]
```

Each token has a high-dimensional representation.

4. Pooling

We ultimately need **one vector representing the entire sentence**.

For example:

Token vectors

↓

[vector for I]

[vector for forgot]

[vector for my]

[vector for password]

↓

Pooling

↓

Sentence embedding

A common approach is **mean pooling**:

```
sentence vector =  
average of the token vectors
```

There are other approaches, depending on the model.

5. Final embedding

Eventually you get something like:

```
"I forgot my password"
```

→

```
[  
  0.124,  
 -0.438,  
  0.871,  
  0.219,  
 -0.153,  
  0.624,  
  ...  
]
```

The actual vector might contain hundreds of dimensions.

For example:

```
384 dimensions
```

means:


```
[0.12, -0.43, 0.87, ...]
```

↑

384 numbers

This is your **sentence embedding**.

6. Why similar meanings become close

Suppose we generate:

```
A = embedding("I forgot my password")
```

```
B = embedding("I cannot remember my login password")
```

Their vectors may look roughly like:

```
A = [0.12, -0.43, 0.87, 0.21, ...]
```

```
B = [0.15, -0.41, 0.84, 0.19, ...]
```

The vectors are close in vector space.

Then:

```
CosineSimilarity(A, B)
```

↓

0.90+

Whereas:

```
C = embedding("Employees can apply for annual leave")
```

could be much farther away:

```
CosineSimilarity(A, C)
```

↓

0.10

The exact scores depend on the model.

7. Your Semantic Search now

You will generate embeddings **once** for your documents:

```
documents.json
  ↓
Document 1 → Embedding
Document 2 → Embedding
Document 3 → Embedding
...
Document 35 → Embedding
```

Store them.

Then whenever a user searches:

```
"I can't remember my password"
```

you only need to generate **one new embedding**:

```
User Query
  ↓
Embedding
  ↓
Query Vector
```

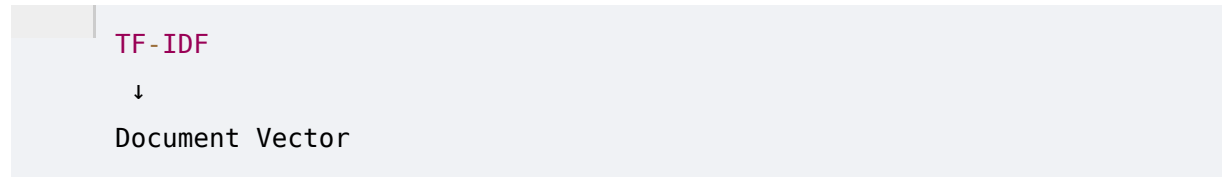
Then:

```
Query Vector
  ↓
Cosine Similarity
  ↓
Document 1
Document 2
Document 3
...
Document 35
  ↓
Sort
  ↓
Top K
```

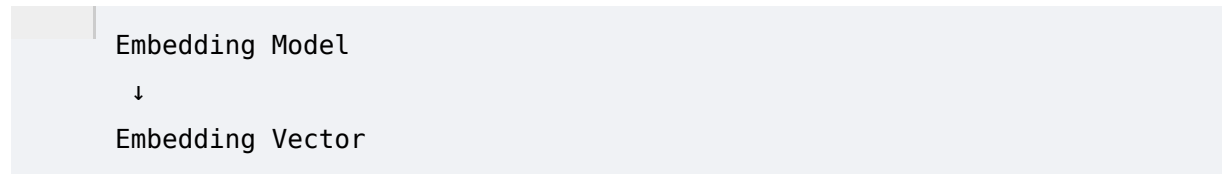
This is much more efficient than generating document embeddings every time someone searches.

8. Important distinction from your Stage 1

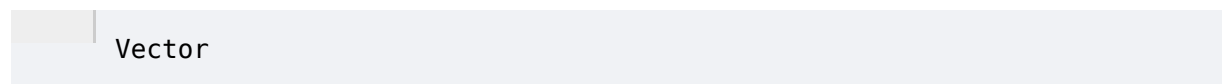
Your Stage 1:



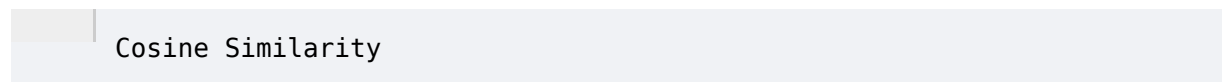
Your Stage 2:



Both eventually give you:

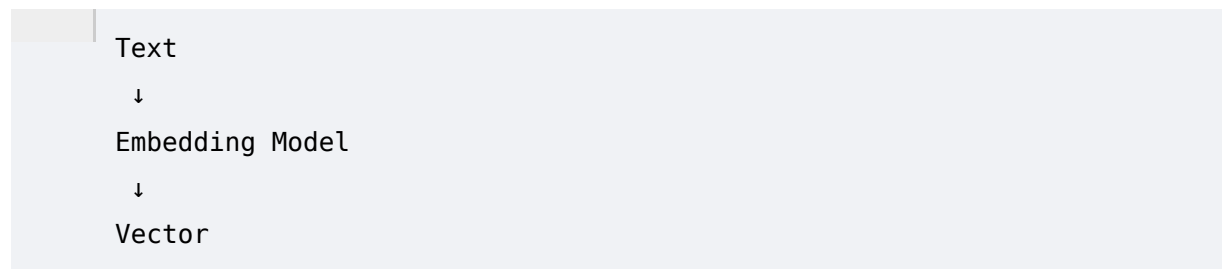


and both can use:



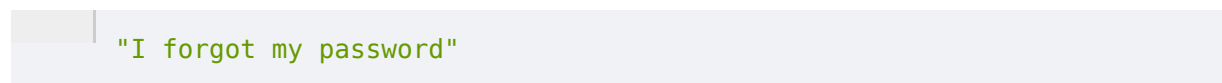
That's why your existing `CosineSimilarity()` method can probably be reused.

The major new component is:



Next step

Now we should **choose and install a real embedding model for your C# project**, then load it and generate your first vector from:



That will be our first actual Stage 2 implementation.

That will be our first actual Stage 2 implementation.

now give me this stage

Yes. Let's implement the **first real Stage 2 embedding** in your C# project.

For a local C# implementation, we'll use **all-MiniLM-L6-v2** in **ONNX format**. Its ONNX model is about 90 MB, and the model is designed for sentence embeddings.

Step 2A — Install ONNX Runtime

In your C# project, install:

```
dotnet add package Microsoft.ML.OnnxRuntime
```

Microsoft's .NET ecosystem supports consuming pretrained ONNX models locally.

Step 2B — Download the model

Use the official Hugging Face model repository:

[all-MiniLM-L6-v2 ONNX model](#)

For our first test, use:

```
model.onnx
```

The repository also contains optimized/quantized variants.

Put it in your project:

```
YourProject
|
|— Models
|   |— all-MiniLM-L6-v2
|       |— model.onnx
|
|— Services
|   |— EmbeddingService.cs
|
|— Program.cs
```

Set the ONNX file properties:

```
Build Action: Content
Copy to Output Directory: Copy if newer
```

Step 2C — Important: the tokenizer

There is one thing we **cannot skip**.

The ONNX model doesn't accept:

```
"I forgot my password"
```

directly.

It expects token IDs and an attention mask.

So the complete implementation is:

```
"I forgot my password"
    ↓
Tokenizer
    ↓
input_ids
attention_mask
    ↓
ONNX Model
    ↓
token embeddings
    ↓
Mean Pooling
    ↓
Normalize
    ↓
384-dimensional embedding
```

The `all-MiniLM-L6-v2` ONNX version produces a **384-dimensional embedding** and has a maximum sequence length of 128 tokens.

Step 2D — Don't use your TF-IDF preprocessing

This is important.

For Stage 1 you probably did:

```
lowercase
    ↓
remove punctuation
    ↓
remove stop words
    ↓
stemming/etc.
```

↓
TF-IDF

For the embedding model, **do not feed your TF-IDF-processed text into the tokenizer.**

Give the embedding model the original text:

```
string text = "I forgot my password";
```

The model's tokenizer is responsible for preparing the text.

So:

```
Stage 1:  
Text → Your preprocessing → TF-IDF  
  
Stage 2:  
Text → Model tokenizer → Embedding
```

Step 2E — Your first embedding test

Ultimately we want an API like:

```
float[] embedding = embeddingService.GenerateEmbedding(  
    "I forgot my password");
```

And the result should conceptually be:

```
Embedding length = 384  
  
[  
    0.0231,  
   -0.0842,  
    0.1173,  
    ...  
]
```

Then:

```
Console.WriteLine(embedding.Length);
```

should give:

Step 2F — One important implementation detail

I don't want you to copy a simplified ONNX example that only calls:

```
session.Run(...)
```

because **the tokenizer and pooling are part of the sentence-transformer pipeline**.

The model repository itself describes the ONNX model as incorporating the Transformer, Pooling, and Normalize components for the sentence embedding pipeline.

For our C# implementation, we'll therefore build the service around:

```
Tokenizer
+
ONNX Runtime
+
Pooling
+
Normalization
```

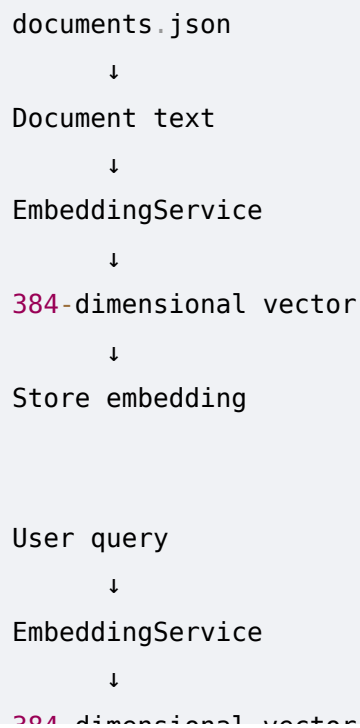
Our target class

We'll make:

```
public class EmbeddingService
{
    public float[] GenerateEmbedding(string text)
    {
        // tokenizer
        // ONNX inference
        // pooling
        // normalization
        // return embedding
    }
}
```

Then your existing semantic search becomes:

STAGE 2



Next, the practical coding step is implementing the tokenizer +

EmbeddingService together. This is the part where model-specific details matter, so we'll do it against `all-MiniLM-L6-v2` rather than using a generic ONNX example.